

# ARIA — Week 4 Evaluation Framework

Mastering Agentic AI Certification · Gen Academy · "Evaluate Your Agent" · June 2026

---

## THE PRIMER

---

I will measure **structured-output success, split correctness, entity correctness, extraction faithfulness, split fidelity, and p50/p90 latency** on ARIA's Orchestrator agent using a golden dataset of 25 cases covering **single-requirement, multi-requirement, vague, entity-confusion, and edge-case scenarios**, with **code-based exact/keyword matching** for 3 metrics and **same-provider LLM-as-judge** for 2 metrics. Pass bar: **≥95% structured-output success, ≥90% entity correctness, ≥0.85 extraction fidelity, p90 latency under 30s, cost not yet trackable**. I will run this in LangSmith and report the delta from **baseline (post-harness-fix run)** to **post-improvement (post-retry-budget-fix run)**.

**Honest scope framing:** ARIA has 5 stages (Orchestrator, Notion, GitHub, Jira, Slack). This evaluation round covers **Orchestrator only**. The other 4 stages have LangSmith tracing but zero golden-dataset quality evaluation — see "What Is Next" at the end of this document.

## THE FRAMEWORK

---

### AGENT UNDER TEST

ARIA's **Orchestrator stage** — the LangGraph node that splits free-text input into individual requirements and extracts structured fields (title, description, acceptance criteria, priority) from each. Chosen over the other 4 stages because it is pure LLM (no side effects, safe to run repeatedly) and is where this project's own history shows a real bug class: confusing two similarly-worded requirements about different subjects.

**User outcome:** A PM/engineer pastes free-text requirements (a Slack message, meeting notes, a bug report) and gets back the correct number of distinct requirements, each attributed to the correct subject, faithfully describing what was actually asked — with no merging, no invented scope. If this is wrong, every downstream artifact (Notion BRD, GitHub tech doc, Jira ticket) is built on a wrong foundation.

---

## METRICS (5 QUALITY + 2 PAIRED)

### METRIC 1

#### **structured\_output\_success**

Did parsing succeed without exhausting retries?

*Code-based*

### METRIC 2

#### **split\_count\_match**

Right number of split-out requirements?

*Code-based / exact match*

### METRIC 3

#### **entity\_keyword\_match**

Each requirement attributed to the right subject?

*Code-based / keyword*

### METRIC 4

#### **extraction\_fidelity**

Title/description/criteria faithfully represent the source. Scale 1-5.

*LLM-as-judge*

### METRIC 5

#### **split\_fidelity**

Full split set covers all expected requirements, no merge/invention. Scale 0-1.

*LLM-as-judge*

### PAIRED METRICS

#### **Latency + Tokens**

p50/p90 latency and avg token count per run.

*Behavior / cost proxy*

**Judge method:** Metrics 1-3 → code-based. Metrics 4-5 → LLM-as-judge using the same Mistral model as production (*mistral-large-latest*, temperature 0.0) with a structured Pydantic grading schema. No human review and no independent (different-provider) judge were used this round — flagged as a real gap.

## GOLDEN DATASET

**25 hand-authored cases** (below the course's 30-50 recommended range — flagged as a gap), stored in `golden_dataset.py` and mirrored into a LangSmith Dataset named *aria-golden-v1*.

**Scenario mix:** Single-requirement (5, 20%) · Multi-requirement (8, 32%) · Vague/underspecified (4, 16%) · Entity-confusion adversarial (3, 12%) · Edge cases — long dumps, contradictions, one-word input, filler speech (5, 20%). One case (*multi-01*) is a real production input reused verbatim from earlier debugging.

**Labeling caveat:** All 25 cases and their expected labels were authored by the AI assistant, not independently hand-labeled or reviewed by a human domain expert before use — a deviation from the course's guidance that should be corrected before treating these as fully trustworthy ground truth.

PASS BAR

- ≥ 95% structured\_output\_success

≥ 90% split\_count\_match

≥ 0.90 split\_fidelity
- ≥ 90% entity\_keyword\_match

≥ 0.85 extraction\_fidelity

< 30s p90 latency (orchestrator only)

**Note:** This pass bar was defined retroactively while writing this report — it did not exist when the evals were run, which is itself a process gap worth naming. Cost target not set (not yet measurable).

INSTRUMENTATION

LangSmith tracing (`LANGCHAIN_TRACING_V2`, project *aria*) wired into both local/eval runs and the live Fly.io production deployment. Each trace covers: full graph run → per-node run → per-LLM-call run, with token usage (prompt/completion/total) and latency. Custom metadata — `submission_id`, `requirement_index`, `requirement_count` — is tagged on every call to tie a trace back to a specific input.

**Not captured:** Per-run dollar cost — LangSmith's Mistral integration did not return pricing metadata, so cost is currently visible only as raw token counts, not dollars.

RESULTS

BASELINE RUN — ORCHESTRATOR-EVAL-2158C182

First run after fixing an eval-harness bug. The very first run (before that fix) scored nonsensically low and is excluded as invalid.

| METRIC                    | BASELINE                         | PASS BAR  | STATUS |
|---------------------------|----------------------------------|-----------|--------|
| structured_output_success | 71%                              | ≥ 95%     | X Fail |
| entity_keyword_match      | 71%                              | ≥ 90%     | X Fail |
| split_count_match         | 88%                              | ≥ 90%     | X Fail |
| split_fidelity            | 0.91                             | ≥ 0.90    | ✓ Pass |
| extraction_fidelity       | 0.67 (16/33 judge calls errored) | ≥ 0.85    | X Fail |
| Latency p50 / p90         | 16.3s / 42.1s                    | p90 < 30s | X Fail |
| Avg tokens / run          | 2,110                            | —         | —      |

## FAILURE ANALYSIS

### #1 — Rate-limit-induced extraction fallback (*dominant, ~30% of cases*)

Under sustained Mistral API load, the extraction call's retry budget exhausted and the code's deliberate fallback path (mark the item "failed", return empty fields) kicked in. This is the single largest cost: every `structured_output_success` and `entity_keyword_match` miss traced back to this.

### #2 — Rate-limit-induced split fallback (*~10% of cases, same root cause*)

Same mechanism one level up — the split call falls back to treating multi-requirement input as one item (e.g. a 5-requirement case collapsed to `split_count: 1`).

### #3 — Eval-infrastructure bug (*not a product bug*)

LLM-judge calls had no retry/timeout protection and the eval harness ran 3 examples concurrently — production never runs more than one concurrent Mistral call, so this stacked more simultaneous load on a shared free-tier quota than the system is designed for, producing a first run with near-zero scores that did not reflect product quality at all.

**Notably absent:** Zero cases of genuine entity confusion, requirement merging, or invented scope — the failure mode this dataset was specifically built to catch never appeared in 75 example-executions across 3 runs.

---

### IMPROVEMENT HYPOTHESES

- Implemented** **Lower eval concurrency to 1 + add retry/timeout to judge calls** — targets failure #3. Predicted: judge errors and harness-induced false failures drop to ~0.
- Implemented** **Bump Orchestrator's own retry budget** from *attempts=2* to *attempts=3* (timeout 25s→14s, still under the 60s node deadline) — targets failures #1/#2. Predicted: *structured\_output\_success* and *split\_count\_match* rise meaningfully.
- Proposed** **Provision a separate Mistral API key for eval/dev traffic** so it stops competing with the live deployment's quota — targets the root cause behind #1/#2 directly, since the in-process retry budget can mitigate but can't fully absorb contention it has no visibility into.
- Proposed** **Independent (different-provider) LLM-as-judge cross-check** on a sample, to calibrate the same-provider judge per the course's human/judge-agreement guidance.

### POST-IMPROVEMENT RUN — ORCHESTRATOR-EVAL-F2B06FF5

| METRIC                    | BASELINE      | POST-IMPROVEMENT          | DELTA         | STATUS |
|---------------------------|---------------|---------------------------|---------------|--------|
| structured_output_success | 71%           | 68%                       | -3pp          | ✗ Fail |
| entity_keyword_match      | 71%           | 68%                       | -3pp          | ✗ Fail |
| split_count_match         | 88%           | 92%                       | +4pp          | ✓ Pass |
| split_fidelity            | 0.91          | 0.95                      | +0.04         | ✓ Pass |
| extraction_fidelity       | 0.67          | 0.60 (8/29 errors, -20pp) | -0.07         | ✗ Fail |
| Latency p50 / p90         | 16.3s / 42.1s | 20.4s / 51.6s             | +4.1s / +9.5s | ✗ Fail |
| Avg tokens / run          | 2,110         | 2,191                     | +4%           | —      |

**Verdict:** *split\_count\_match* and *split\_fidelity* now pass. *structured\_output\_success*, *entity\_keyword\_match*, *extraction\_fidelity*, and p90 latency still fail — driven by the same ambient quota-contention issue that hypothesis #3 (not yet implemented) targets directly. Higher latency is expected: more retry attempts cost time.

# WHAT IS NEXT

---

**Top remaining failure mode:** Mistral API quota shared between eval traffic and the live Fly deployment — the retry-budget fix helped the metrics it could reach but can't eliminate contention it has no visibility into.

- Separate API key for eval traffic (hypothesis #3) — directly eliminates the root cause of the dominant failure mode
- Build the curated full-pipeline subset to extend evaluation to Notion/GitHub/Jira/Slack (currently zero quality coverage on those 4 stages)
- Human pass over the golden dataset's labels, per the labeling caveat documented above
- Wire cost tracking (manual token × price calculation, since LangSmith isn't returning it automatically)
- Wire this as a recurring CI gate instead of a manual on-demand run

**Production monitoring strategy:** Alert if `structured_output_success` (sampled from live traces, not just eval) drops >10pp week-over-week, or if p95 latency exceeds the existing 60s node deadline on >5% of runs.

# SUBMISSION PACKAGE

---

|                                                                                                |                                                                        |
|------------------------------------------------------------------------------------------------|------------------------------------------------------------------------|
| <code>golden_dataset.py</code><br>Python · mirrored to LangSmith Dataset <i>aria-golden-v1</i> | <code>evaluators.py</code><br>Evaluator code                           |
| <code>METRICS.md</code><br>Metric definitions                                                  | <code>EVALUATION_REPORT.md</code><br>Narrative failure-analysis report |

**LangSmith project:** *aria* (same project used by both production tracing and eval experiments). **Loom walkthrough:** not yet recorded — this document and `EVALUATION_REPORT.md` are the intended script.